

Forking for Number Questions

Each voter i submits the following tuple of information:

$$(a_i, l_i, u_i, r_i^0) \in \mathbb{R}^3 \times [0, 1]$$

such that $a_i \in [l_i, u_i]$. Essentially, a_i represents the value that the voter i thinks is the correct answer, and l_i and u_i represent lower and upper bounds respectively for what she thinks is a reasonable answer. Then, r_i^0 represents the threshold of tokens on a minority fork at which i is willing to participate.

To compute the winning option on the main fork a_{main} , various mechanisms are possible (essentially this is the same space of mechanism design as what voting system to use for number-valued question dispute kits in general). The simplest approach is to output as the answer the median of the a_i .

Then, to determine what forks should be generated, one can do the following:

1. Take $\mathcal{I}$ to be the set of token holders.
2. For all $i \in \mathcal{I}$, compute $r_i = r_i^0 \cdot \frac{\text{total number}}{\text{of tokens}}$.
3. All token holders i such that $a_{\text{main}} \in [l_i, u_i]$ remain on the fork where a_{main} wins. Remove these holders i from $\mathcal{I}$.
4. Define the set $X = \{l_i : i \in \mathcal{I}\} \cup \{u_i : i \in \mathcal{I}\}$.
5. For all $x \in X$,
 - Take the list L_x of voters i such that $x \in [l_i, u_i]$ and $i \in \mathcal{I}$.
 - Take the sum of the number of tokens over all voters on L_x . Denote this by s_x .
6. For each value $x \in X$:
 - Sort L_x by the user's r_i .
 - Take the user i with the largest r_i . Compare r_i to s_x . If $r_i > s_x$, remove i from L_x .
 - Recalculate s_x as the sum of the number of tokens over all voters on L_x .
 - Repeat the two previous steps until L_x stabilizes.
7. Then, over the possible choices x , take the option that maximizes s_x and for which $L_x \neq \emptyset$.
8. All remaining token holders on L_x go to an alternative fork where x is the winning value. Remove these token holders from $\mathcal{I}$.
9. Repeat steps 4-8 until there is no x produced such that $L_x \neq \emptyset$. This determines if there are still compatible forks possible.

0.0.1 Properties of Forking Proposal

Proposition 1. *The above process determines the single largest possible fork.*

Proof. To fill in. Similar argument to the ranked vote case in the yellow paper. Note that, as the intervals at which the token holders approve are closed intervals, the value that maximizes compatible fork choice must be realized at one of the endpoints of these intervals, so it suffices to consider only these values. □

1 Optimistic version

Suppose there are M voters. The system should have in its state information on the various voters and how they voted. Namely, there should be the following vectors of length M :

- l , where $l_i = l[i]$ is the lower bound provided by the voter,
- u , where $u_i = u[i]$ is the upper bound provided by the voter,
- r , where $r_i = r[i]$ is the threshold provided by the voter, and
- $stake$, where $stake[i]$ is the number of tokens held by the user i .

The values of i should be some identifier for the voter, such as numbering the voters $i = 0, \dots, M - 1$.

The system should also compute the winning outcome a_{main} before the submission period for the optimistic forking process begins.

We further suppose that the system keeps in its state the following information provided by submitters during the optimistic process:

- x , a vector of length k , where $x_j = x[j] \in \mathbb{R}$ represents a possible outcome to the dispute other than a_{main} .
- s , a vector of length k , where $s_j = s[j] \in \mathbb{N}$ represents the total number of tokens that goes to the fork where x_j is considered the correct outcome.
- A , an array of size $m \times 2$, where the first column gives the identifiers of voters who go to some fork. Namely, $A[j][1] \in \{0, \dots, M - 1\}$ for each j , so that we can find the lower and upper bounds, etc of the corresponding voter by taking $l[A[j][1]]$, $u[A[j][1]]$, etc. The second column indicates where the voter $A[j][1]$ should be sent at the conclusion of the forking process. For example, $A[j][1]$ would be sent to a fork where the value $x[A[j][2]]$ is recognized as having won.

This state is initialized to $\emptyset$ (and $\emptyset$ is potentially the correct outcome). Note, particularly, the values of k and m which represent the number of (non-main) forks to create and the number of voters to send to a (non-main) fork respectively, will vary from one submission to another. Both of these values are upper bounded by M . A challenge takes the following form:

$$x', s', A',$$

where x' and s' are of length l , and A' is of size $b \times 2$.

A deposit with the challenge is not necessary, as the contract should be able to determine in the execution of the challenge transaction whether the challenge is superior to the pending forks in the state. Thus, the challenger just needs to pay the gas for that transaction.

Then, the challenge transaction should perform checks corresponding to the following pseudocode:

```

# Phase 1 - formatting checks on  $s'$ 
For  $t = 0, \dots, l - 2$ 
    If  $s'[t] < s'[t + 1]$  # Check that  $s'_t$  are in decreasing order
        halt, revert
    If  $s'[t] \leq 0$ 
        halt, revert

# Phase 2 - (lexicographic) size improvement over pending forks
successSize = 0
 $t = 0$ 
While successSize = 0 AND  $t \leq \min\{k, l\}$ 
    #Attention to indexes;  $t$  index of  $s$  so need to be in range. But also need to checks for cases  $k, l = 0$ .
    If  $t \leq l - 1$  AND  $t > k - 1$ 
        successSize = 1 # Equal on all forks so far, but challenge has more forks.
    Else If  $t > l - 1$  # Equal on all forks so far, but challenge has fewer forks.
        halt, revert
    Else
        If  $s'[t] > s[t]$  # Checking that produces a larger fork before any eventual smaller forks.
            successSize = 1 #Note, using  $s'_t$  in decreasing order here.
        If  $s'[t] < s[t]$ 
            halt, revert
     $t = t + 1$ 
If successSize = 0 # Can get through prev. loop with successSize=0 if forks sizes equal, no improvement
    halt, revert

# Phase 3 - check that each voters conditions met
ForkSizes initialized to vector of zeros of length  $l$ .
For  $j = 0, \dots, b$  # Looping over voters to be sent to forks.
     $i = A'[j][1]$  # Id of voter.
     $t = A'[j][2]$  # Fork index.
    If  $i < 0$  OR  $i > M - 1$ 
        halt, revert, # voter id out of range.
    If  $t < 0$  OR  $t > l - 1$ 
        halt, revert, # fork id out of range.
    If  $j \geq 1$  AND  $i < A'[j - 1][1]$ 
        halt, revert, # This check ensures ids increasing so don't count same voter twice.
    If  $a_{\text{main}} \in [l[i], u[i]]$ 
        halt, revert # Voter should go to main fork.
    If  $s'[t] \geq r[i]$  AND  $x'[t] \in [l[i], u[i]]$ 
        # I.e.  $s'_t \geq r_i$  AND  $x'_t \in [l_i, u_i]$ , so fork compatible for voter.
        ForkSizes[t] = ForkSizes[t] + stake[i].
    Else
        halt, revert # Fork incompatible for voter.

```

Phase 4 - check fork sizes match

For $t = 0, \dots, l$

 If $ForkSizes[t] \neq s'[t]$

 halt, revert, # Sizes of forks don't match claim, invalidating checks above.

Replace (x, s, A) in state with (x', s', A') # Made it through all checks.

Then, after the challenge period is over, one can execute the forks by performing:

For $j = 1, \dots, b$

 Send voter $A[j][1]$ to fork where $x[A[j][2]]$ is winner

The challenge and execute transactions require $O(M)$ steps. Note that these steps involve referencing entries on arrays that are $O(\text{number of forks})$, so care should be taken in how memory is stored and accessed so that this does not create dependance on the number of forks.